МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИКН)

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Курс «Сетевые технологии»

Отчёт по лабораторной работе на тему:

«ЛР-4. Packet Tracer. Настройка начальных параметров коммутатора»

Выполнил: студент группы БИВТ-24-7

Бут Иван Антонович

Проверил: КТН, СНС, доцент

Микитенко Игорь Иванович

МОСКВА 2026

Цель работы

Освоить базовую настройку коммутатора Cisco: проверить конфигурацию по умолчанию, защитить доступ к консоли и привилегированному режиму, включить шифрование открытых паролей, настроить баннер MOTD и сохранить конфигурацию в энергонезависимой памяти.

Основные задачи лабораторной работы

1) Проверить конфигурацию коммутатора по умолчанию и ответить на вопросы по выводу show running-config.
2) Настроить имя устройства, пароль консоли, enable password и enable secret на коммутаторе S1.
3) Включить шифрование открытых паролей и настроить баннер MOTD.
4) Сохранить running-config в startup-config и проверить содержимое NVRAM.
5) Повторить ту же последовательность действий на коммутаторе S2.

Схема стенда

Для основной части лабораторной работы использовались два коммутатора Cisco и два персональных компьютера. На дополнительном этапе я уже работал с этой же схемой как с маленьким стендом для VLAN и trunk, поэтому ее логично показать сразу.

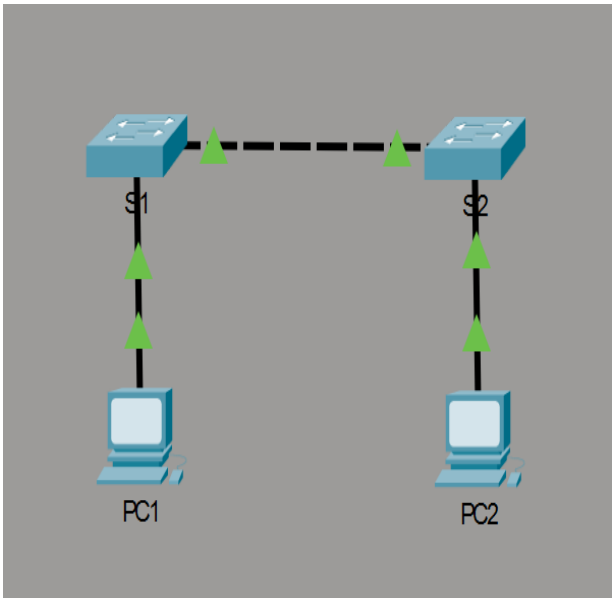

Рисунок 1 - Схема подключения устройств в Packet Tracer

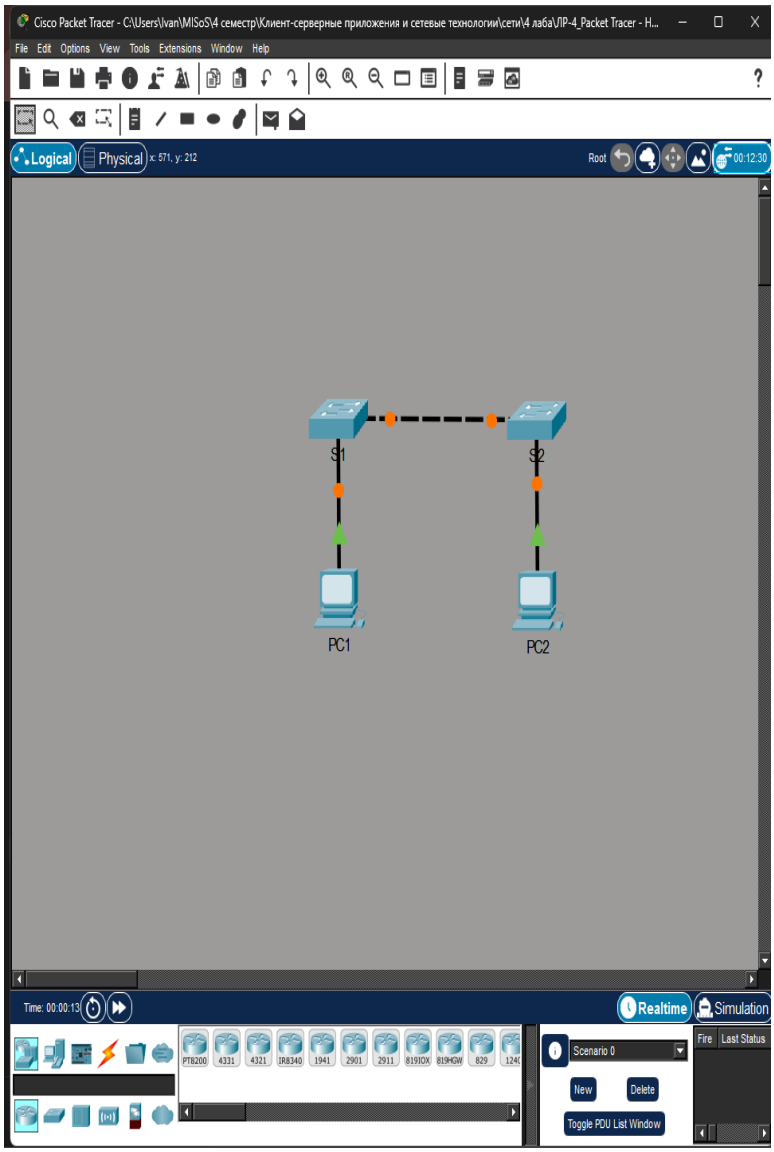

Рисунок 2 - Рабочее окно Cisco Packet Tracer с собранной топологией

МОСКВА 2026

Ход выполнения работы

Часть 1. Проверка конфигурации коммутатора по умолчанию

По этому пункту нужно было войти в привилегированный режим EXEC, посмотреть текущую конфигурацию и по ней ответить на контрольные вопросы. Я начал с S1 и сначала специально посмотрел именно "чистое" состояние устройства, без своих настроек.

Логи

```
Switch>enable
Switch#show running-config
Building configuration...

Current configuration : 1080 bytes
version 12.2
no service timestamps log datetime msec
no service timestamps debug datetime msec
no service password-encryption
hostname Switch

interface FastEthernet0/1
interface FastEthernet0/2
interface FastEthernet0/3
...
interface FastEthernet0/24
interface GigabitEthernet0/1
interface GigabitEthernet0/2
interface Vlan1
 no ip address
 shutdown

line con 0
line vty 0 4
 login
line vty 5 15
 login
end
```

Из этого вывода видно главное. У коммутатора по умолчанию еще нет имени S1 или S2, парольная защита не настроена, а startup-config в NVRAM еще не создан. При этом уже присутствуют линии удаленного доступа vty и базовые интерфейсы.

На вопросы по первой части я ответил так. Интерфейсов FastEthernet у коммутатора 24. Интерфейсов GigabitEthernet 2. Диапазон vty-линий 0-15, он показан двумя блоками: line vty 0 4 и line vty 5 15. Команда для просмотра содержимого NVRAM: show startup-config. Сообщение "startup-config is not present" появляется потому, что конфигурация пока существует только в оперативной памяти и ни разу не сохранялась.

По первой части у меня получилась хорошая стартовая точка: я увидел исходное состояние устройства и дальше уже было легко отслеживать, какие строки реально появляются после каждой новой команды.

Часть 2. Настройка основных параметров коммутатора S1

Во второй части я уже пошел по шагам настройки. Сначала присвоил имя устройству, потом защитил консоль, после этого настроил enable password и enable secret. Чтобы не смешивать все в один сплошной кусок, логи разбил на несколько этапов.

Шаг 1. Назначение имени устройства и защита консоли

Здесь команда hostname нужна, чтобы в приглашении CLI сразу было видно, с каким устройством я работаю. Дальше line console 0, password и login включают обычную аутентификацию при входе через консоль.

Логи

```
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#exit
S1#

S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#line console 0
S1(config-line)#password letmein
S1(config-line)#login
S1(config-line)#exit
S1(config)#exit
S1#
exit

S1 con0 is now available
Press RETURN to get started.

User Access Verification
Password:
```

Когда я вышел и заново зашел на консоль, устройство уже запросило пароль. Это и есть практическое подтверждение того, что команда login сработала как надо. Без нее пароль был бы задан в конфигурации, но реально не применялся бы при подключении.

Шаг 2. Настройка привилегированного режима

Теперь я добавил enable password, а затем более защищенный enable secret. Здесь интересно то, что enable secret хранится в виде хэша и имеет приоритет над обычным enable password.

Логи

```
S1>enable
S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#enable password c1$c0
S1(config)#exit
S1#

S1>enable
Password:
S1#show running-config
Building configuration...

hostname S1
enable password c1$c0

line con 0
 password letmein
 login
line vty 0 4
 login
line vty 5 15
 login
end
```

В этот момент в running-config пароль enable password еще отображается открытым текстом. Именно поэтому следующим шагом я добавил enable secret, а потом включил шифрование открытых паролей.

Логи

```
S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#enable secret itsasecret
S1(config)#exit
S1#
S1#show running-config
Building configuration...

hostname S1
enable secret 5 $1$mERr$ILwq/b7kc.7X/ejA4Aosn0
enable password c1$c0

line con 0
 password letmein
 login
end

S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#service password-encryption
S1(config)#exit
S1#
show run
Building configuration...

service password-encryption
hostname S1
enable secret 5 $1$mERr$ILwq/b7kc.7X/ejA4Aosn0
enable password 7 08221D0A0A49
line con 0
 password 7 082D495A041C0C19
 login
```

После service password-encryption открытые пароли уже записываются в конфигурацию в зашифрованном виде типа 7. При этом enable secret и до

этого не был виден открытым текстом, потому что IOS хранит его как хэш. На этом этапе я уже получил нормальную базовую защиту для S1.

Если отвечать совсем по сути, то разница между enable password и enable secret здесь видна сразу на практике. enable password без шифрования отображается как обычная строка. enable secret хранится в виде хэша. После service password-encryption пароли на линиях и enable password тоже перестают быть открытыми в show run.

Часть 3. Настройка баннера MOTD

Следующий пункт очень короткий по команде, но полезный по смыслу. Баннер MOTD нужен не ради красоты, а как предупреждение для любого пользователя, который подключается к устройству.

Логи

```
S1>enable
Password:
S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#banner motd "This is a secure system. Authorized Access Only!"
S1(config)#exit
S1#
%SYS-5-CONFIG_I: Configured from console by console
```

Эта команда задает текст сообщения, которое будет показано при входе на устройство. В рамках лабораторной смысл MOTD вполне прямой: предупредить, что доступ разрешен только авторизованным пользователям. В реальной сети это еще и важный юридический момент, потому что баннер прямо фиксирует запрет на несанкционированный доступ.

По этой части у меня все получилось без проблем. Баннер настроен, в конфигурации он появился, и дальше при подключении к устройству его можно увидеть до начала полноценной работы в CLI.

Часть 4. Сохранение конфигурации в NVRAM

После настройки S1 нужно было сохранить результат в энергонезависимой памяти. Тут важна простая мысль: пока конфигурация лежит только в running-config, она исчезнет после перезагрузки или выключения питания.

Логи

```
copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]

S1#copy run start
Destination filename [startup-config]?
Building configuration...
[OK]

S1#show startup-config
Using 1252 bytes

service password-encryption
hostname S1
enable secret 5 $1$mERr$ILwq/b7kc.7X/ejA4Aosn0
enable password 7 08221D0A0A49

interface Vlan1
 no ip address
 shutdown

banner motd ^CThis is a secure system. Authorized Access Only!^C

line con 0
 password 7 082D495A041C0C19
 login
line vty 0 4
 login
line vty 5 15
 login
```

Здесь видно сразу две полезные вещи. Во-первых, длинная команда copy running-config startup-config действительно может быть сокращена до copy run start. Во-вторых, show startup-config уже показывает сохраненную конфигурацию, а значит изменения точно записаны в NVRAM.

Именно после этого шага стало понятно, что настройка закончена не "на экране", а реально закреплена в памяти коммутатора.

Часть 5. Настройка коммутатора S2

На S2 я повторил ту же самую логику, но уже без долгого разгона, потому что все основные команды были отработаны на S1. При этом именно на втором устройстве хорошо видно, как баннер появляется при новом входе, а show run подтверждает шифрование паролей.

Логи

```
Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S2
S2(config)#exit
S2#

S2#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S2(config)#line console 0
S2(config-line)#password letmein
```

```
S2(config-line)#login
S2(config-line)#exit
S2(config)#exit
S2#

S2#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S2(config)#enable password c1$c0
S2(config)#enable secret itsasecret
S2(config)#exit
S2#

S2#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S2(config)#banner motd "Authorized access only. Unauthorized access is prohibited
and violators will be prosecuted to the full extent of the law."
S2(config)#service password-encryption
S2(config)#exit
S2#
```

После этого я проверил конфигурацию и отдельно увидел поведение устройства при новом входе. Это полезный момент, потому что баннер здесь уже показан не просто как строка команды, а как реально работающее сообщение перед аутентификацией.

Логи

```
S2#show run
Building configuration...

service password-encryption
hostname S2
enable secret 5 $1$mERr$ILwq/b7kc.7X/ejA4Aosn0
enable password 7 08221D0A0A49
...

S2#exit

S2 con0 is now available
Press RETURN to get started.

Authorized access only. Unauthorized access is prohibited and violators will be
prosecuted to the full extent of the law.

User Access Verification
Password:

S2>enable
Password:
S2#show run
```

Чтобы закрыть пункт до конца, конфигурацию на S2 тоже нужно сохранить. По журналу команд это делается той же самой командой copy running-config startup-config. После сохранения show startup-config уже показывает startup-config в памяти NVRAM.

Логи

```
S2#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]

S2#show startup-config
Using 1325 bytes
```

```
service password-encryption
hostname S2
enable secret 5 $1$mERr$ILwq/b7kc.7X/ejA4Aosn0
enable password 7 08221D0A0A49
...
line con 0
 password 7 082D495A041C0C19
 login
```

По итогам пятой части S2 настроен аналогично S1: имя устройства задано, пароль консоли работает, enable password и enable secret заданы, баннер отображается, шифрование включено, а конфигурация сохранена.

Дополнительное исследование

Для дополнительного балла я сделал отдельный мини-эксперимент с VLAN и trunk. Здесь уже было не просто повторение базовой настройки, а реальная проверка того, как коммутаторы ведут себя при ошибке в назначении портов и как это влияет на обмен трафиком между ПК.

Эксперимент. Создание VLAN 10 и настройка trunk между S1 и S2

Цель эксперимента: создать **VLAN 10**, назначить пользовательские порты в access, настроить trunk на межкоммутаторном линке и проверить обмен трафиком между ПК.

Сначала я создал VLAN 10 на обоих коммутаторах. На этом этапе все выглядело прямолинейно: создаем VLAN, даем ему имя USERS и назначаем порт, куда подключен ПК.

Логи

```
S1>enable
Password:
S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#vlan 10
S1(config-vlan)#name USERS
S1(config-vlan)#exit
S1(config)#interface fastEthernet0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#exit
S1(config)#exit
S1#

S1#show vlan brief

VLAN Name                             Status    Ports
1    default                          active    Fa0/2, Fa0/3, Fa0/4, ..., Gig0/1,
Gig0/2
10   USERS                            active    Fa0/1
```

Дальше я проверил trunk и тут уже началась самая интересная часть. По выводу стало видно, что межкоммутаторный линк использует Fa0/1, а не тот порт, который я сначала мысленно отвел под ПК. Из-за этого на одной стороне порт фактически работал как trunk, а на другой еще оставался access VLAN 10. Сразу прилетели предупреждения CDP и STP.

Логи

```
S1#show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Fa0/1       on           802.1q         trunking      1

Port        Vlans allowed on trunk
```

```
Fa0/1       10

S1#
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/1
(1), with S2 FastEthernet0/1 (10).
```

На S2 ошибка проявилась еще нагляднее. Сначала я попробовал сделать trunk на GigabitEthernet0/1, но по факту реальный линк между коммутаторами был на FastEthernet0/1. Из-за этого и появились сообщения про Native VLAN mismatch и блокировку порта в STP.

Логи

```
S2#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S2(config)#vlan 10
S2(config-vlan)#name USERS
S2(config-vlan)#exit
S2(config)#interface fastEthernet0/1
S2(config-if)#switchport mode access
S2(config-if)#switchport access vlan 10
S2(config-if)#exit
S2(config)#interface gigabitEthernet0/1
S2(config-if)#switchport mode trunk
S2(config-if)#switchport trunk allowed vlan 10
S2(config-if)#exit
S2(config)#exit
S2#

%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on FastEthernet0/1
(10), with S1 FastEthernet0/1 (1).
%SPANTREE-2-RECV_PVID_ERR: Received 802.1Q BPDU on non trunk FastEthernet0/1
VLAN1.
%SPANTREE-2-BLOCK_PVID_LOCAL: Blocking FastEthernet0/1 on VLAN0001. Inconsistent
port type.
```

После этого я уже исправил распределение ролей портов. Пользовательские ПК перенес на Fa0/2, а Fa0/1 на обеих сторонах перевел именно в trunk. Это как раз тот случай, когда ошибка оказалась полезной: из-за нее я не просто "вбил команды", а увидел, почему важно сначала понять физическую схему и только потом назначать режимы портов.

Логи

```
S1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#end
S1#

S2#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
S2(config)#interface fa0/2
S2(config-if)#switchport mode access
S2(config-if)#switchport access vlan 10
S2(config-if)#interface fa0/1
S2(config-if)#switchport mode trunk
S2(config-if)#switchport trunk allowed vlan 10
S2(config-if)#end
S2#
```

На ПК я задал **IP-адреса** из одной подсети: PC1 получил **192.168.10.2**, а PC2 получил **192.168.10.3**. После этого сделал обычную проверку ping.

Логи

```
C:\>ping 192.168.10.3

Pinging 192.168.10.3 with 32 bytes of data:

Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128
Reply from 192.168.10.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.10.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

В итоге эксперимент дал мне сразу несколько понятных выводов. Во-первых, VLAN 10 реально работает только тогда, когда access-порты и trunk назначены на правильные физические интерфейсы. Во-вторых, CDP и STP очень быстро показывают несогласованность настроек. В-третьих, успешный ping между ПК окончательно подтвердил, что трафик VLAN 10 пошел через межкоммутаторный линк так, как и должен.

Вопросы безопасности по дополнительному заданию

1. VLAN дает логическую сегментацию сети второго уровня, но это не полноценная защита само по себе. Если между VLAN нужна реальная фильтрация, уже подключаются ACL или межсетевой экран.

2. Ошибки в trunk-настройке опасны не только с точки зрения "не работает", но и с точки зрения безопасности. Если не ограничивать список разрешенных VLAN, можно получить лишний трафик на магистральном линке.

3. Пользовательские порты лучше держать строго в режиме access. Это снижает риск неверного согласования режима порта и убирает лишнюю неоднозначность при диагностике.

4. Сообщения Native VLAN mismatch и ошибки STP в этой работе показали, что даже маленькая путаница с портами быстро приводит к блокировке линка. То есть проверять show vlan brief и show interfaces trunk действительно нужно, а не "для галочки".

Общий вывод по работе

По первой части я зафиксировал исходное состояние коммутатора и на практике увидел, что в конфигурации по умолчанию нет ни сохраненного startup-config, ни парольной защиты. По второй части стало понятно, как последовательно строится базовая защита устройства: сначала console password, потом enable password, потом enable secret и в конце шифрование открытых паролей. На третьем шаге баннер MOTD показал, как даже одна короткая команда добавляет осмысленное предупреждение для любого пользователя, который подключается к устройству.

Четвертая часть закрыла самый "жизненный" момент: если не выполнить copy running-config startup-config, то вся настройка остается временной. Пятая часть подтвердила, что та же схема спокойно воспроизводится и на втором коммутаторе. То есть после S1 я уже не просто повторял команды, а понимал, зачем каждая из них нужна и что именно она меняет в running-config и startup-config.

Дополнительный эксперимент с VLAN оказался самым полезным с практической точки зрения. Я сначала ошибся с ролью портов, получил Native VLAN mismatch и сообщения STP, а потом уже осознанно исправил схему: access для ПК, trunk для линка между коммутаторами. Успешный ping между 192.168.10.2 и 192.168.10.3 показал, что после исправления топология действительно заработала. Для меня это был хороший пример того, как даже простая лабораторная быстро перестает быть "набором команд", если начинаешь разбирать, что именно происходит на портах и почему.